

Relatório Protótipo 02 – Sistema embarcado distribuído de iluminação inteligente

Nesta etapa, fizemos alterações no código do Protótipo 01 para que ele pudesse se conectar ao broker público do HiveMQ e publicar mensagens em um tópico. O tópico é especificado diretamente no código MicroPython, rodado pelo ESP32 que também é conectado ao protoboard para detectar o nível de luminosidade e presença no ambiente – a lógica de mudança de estados permanece inalterada em relação ao código do Protótipo 02.

Mudanças feitas no código a serem destacadas:

- Adição de seção para configuração do Wi-Fi e constantes MQTT;
- Funções para conexão com a rede Wi-Fi fornecida durante as aulas da disciplina e conexão MQTT com o broker público;
- Função para publicar evento no broker público em tópico especificado por meio do código. Faz uso de try/except para checar se a conexão com o broker está em pé – se não estiver, tenta reconexão repetidamente.

Vídeo do funcionamento conjunto de todos os módulos (ESP32 + código + monitoramento do tópico pelo broker) está anexo à entrega da atividade no Classroom. É exibida abaixo a versão final do código MicroPython referente ao Protótipo 2, seguida por uma captura de tela da interface do <https://www.hivemq.com/demos/websocket-client/> recebendo os eventos publicados no tópico.

Python

```
from machine import Pin, ADC, time_pulse_us
import network
import time
import ujson
from umqtt.simple import MQTTClient

# ----- Configuração dos pinos -----
# Acho que não mudamos nada aqui na última aula... confirmar please
TRIG_PIN = 5
ECHO_PIN = 18
LED_PIN = 4
LDR_PIN = 34

LIMIAR_DISTANCIA_CM = 50
LIMIAR_LUMINOSIDADE = 1500

trig = Pin(TRIG_PIN, Pin.OUT)
echo = Pin(ECHO_PIN, Pin.IN)
led = Pin(LED_PIN, Pin.OUT)

ldr = ADC(Pin(LDR_PIN))
ldr.atten(ADC.ATTN_11DB)
ldr.width(ADC.WIDTH_12BIT)

# ----- Configuração Wi-Fi -----
WIFI_SSID = "caetano"
```

```
WIFI_SENHA = "ubi6206comp"
```

```
# ----- Configuração MQTT -----
```

```
MQTT_BROKER = "broker.hivemq.com"
```

```
MQTT_PORTA = 1883
```

```
MQTT_CLIENT_ID = "esp32_grupo1_caetanoubicomp" # deve ser único no broker
```

```
MQTT_TOPICO = "pervasiva_grupo1/iluminacao"
```

```
# ----- Conexão Wi-Fi -----
```

```
def conectar_wifi():
```

```
    wlan = network.WLAN(network.STA_IF)
```

```
    wlan.active(True)
```

```
    wlan.connect(WIFI_SSID, WIFI_SENHA)
```

```
    print("Conectando ao Wi-Fi", end="")
```

```
    while not wlan.isconnected():
```

```
        print(".", end="")
```

```
        time.sleep(500)
```

```
    print("\nWi-Fi conectado:", wlan.ifconfig()[0])
```

```
# ----- Conexão MQTT -----
```

```
def conectar_mqtt():
```

```
    cliente = MQTTClient(MQTT_CLIENT_ID, MQTT_BROKER, MQTT_PORTA)
```

```
    cliente.connect()
```

```
    print("MQTT conectado ao broker:", MQTT_BROKER)
```

```
    return cliente
```

```
# ----- Função de medição de distância -----
```

```
def medir_distancia():
```

```
    trig.off()
```

```
    time.sleep_us(2)
```

```
    trig.on()
```

```
    time.sleep_us(10)
```

```
    trig.off()
```

```
    duracao_us = time_pulse_us(echo, 1, 30000)
```

```
    if duracao_us < 0:
```

```
        return None
```

```
    return duracao_us / 58
```

```
# ----- Função de leitura de luminosidade -----
```

```
def medir_luminosidade():
```

```
    leituras = [ldr.read() for _ in range(5)]
```

```
    return sum(leituras) // len(leituras)
```

```
# ----- Função de decisão de acionamento -----
```

```
def avaliar_contexto(distancia, luminosidade):
```

```

presenca = distancia is not None and distancia < LIMAR_DISTANCIA_CM
escuro   = luminosidade < LIMAR_LUMINOSIDADE
return presenca and escuro

# ----- Função de publicação MQTT -----
def publicar_evento(cliente, estado_led, distancia, luminosidade):
    payload = ujson.dumps({
        "led"       : estado_led,          # "on" ou "off"
        "distancia" : distancia,           # float em cm, ou None
        "luminosidade": luminosidade       # int 0-4095
    })

    try:
        cliente.publish(MQTT_TOPICO, payload)
        print("Publicado:", payload)
    except Exception as e:
        print("Erro ao publicar, tentando reconectar:", e)
        try:
            cliente.connect()
        except:
            print("Reconexão falhou, continuando...")

# ----- Inicialização -----
conectar_wifi()
cliente_mqtt = conectar_mqtt()

# estado anterior do LED - controla quando publicar
# começa como None para forçar publicação na primeira iteração
estado_anterior = None

# ----- Loop principal -----
while True:
    distancia    = medir_distancia()
    luminosidade = medir_luminosidade()

    if distancia is not None:
        dist_str = "{:.1f} cm".format(distancia)
    else:
        dist_str = "fora de alcance"

    print("Distância: {} | Luminosidade: {}".format(dist_str, luminosidade))

    # avalia se o LED deve estar ligado
    deve_ligar = avaliar_contexto(distancia, luminosidade)
    estado_atual = "on" if deve_ligar else "off"

    # aciona o LED
    led.value(1 if deve_ligar else 0)

    # publica APENAS se houve transição de estado

```

```

if estado_atual != estado_anterior:
    print(">> Transição detectada: {} -> {}".format(estado_anterior,
estado_atual))
    publicar_evento(cliente_mqtt, estado_atual, distancia, luminosidade)
    estado_anterior = estado_atual

time.sleep_ms(200)

```

Connection

Host

broker.hivemq.com

Port

8884

ClientID

clientId-lbek2ysi35

Disconnect

Username

Password

Keep Alive

60

SSL

×

Clean Session

×

Last-Will Topic

Last-Will QoS

0

Last-Will Retain

Last-Will Message

Publish

Messages

2026-07-07 20:31:09	Topic: pervasiva/grupo1/iluminacao	Qos: 0	{"led": "off", "luminosidade": 2160, "distancia": 74.655176}
2026-07-07 20:31:07	Topic: pervasiva/grupo1/iluminacao	Qos: 0	{"led": "on", "luminosidade": 650, "distancia": 3.3448276}
2026-07-07 20:31:06	Topic: pervasiva/grupo1/iluminacao	Qos: 0	{"led": "off", "luminosidade": 767, "distancia": null}
2026-07-07 20:31:06	Topic: pervasiva/grupo1/iluminacao	Qos: 0	{"led": "on", "luminosidade": 667, "distancia": 3.3275862}
2026-07-07 20:31:06	Topic: pervasiva/grupo1/iluminacao	Qos: 0	{"led": "off", "luminosidade": 759, "distancia": null}
2026-07-07 20:31:05	Topic: pervasiva/grupo1/iluminacao	Qos: 0	{"led": "on", "luminosidade": 618, "distancia": 3.6551724}
2026-07-07 20:31:05	Topic: pervasiva/grupo1/iluminacao	Qos: 0	{"led": "off", "luminosidade": 814, "distancia": null}

Subscriptions

Add New Topic Subscription

Qos: 2

pervasiva/grupo1/ilu...

X